

Hybrid Methods

Learning Objectives

By the end, you can:

1. Design hybrid ML pipelines: **encode** → **PQC** → **decode** → **classical**.
2. Choose encoding/ansatz/decoding strategies under qubit and depth budgets.
3. Train with **parameter-shift** / **SPSA**, using batching, caching, and mitigation.
4. Embed **quantum layers** as drop-in modules inside MLPs/CNNs (PyTorch, TF/Keras, PennyLane).
5. Debug trainability (barren plateaus) and hardware issues (latency, readout bias).

Motivation

- **NISQ reality:** few qubits; noise limits circuit depth.
- **Hybrid split:** quantum does compact, entangling feature maps / similarity; classical does scale, data I/O, heavy optimization.
- **Win:** shorter circuits, smaller shot budgets, better gradient SNR, simpler deployment.

Hybrid Interface Pattern

Pipeline:

$$x \in \mathbb{R}^d \xrightarrow{U_\phi(x)} |0\rangle^{\otimes n} \xrightarrow{U(\theta)} |\psi\rangle \xrightarrow{\text{measure}} z \in \mathbb{R}^m \xrightarrow{\text{classical}} \hat{y}$$

Encoding (Data $\rightarrow$ Circuit)

- **Angle:** $R_Y(\alpha x_i), R_Z(\beta x_i)$ (low depth; ≈ 1 qubit/feature).
- **Entangling angle:** ZZFeatureMap (captures pairwise interactions; adds 2-qubit gates).
- **Amplitude:** $|x\rangle/\|x\|$ (compact; costly state-prep).
- **Data re-uploading:** repeat $U_\phi(x)$ between trainable layers for expressivity.

Tip: Standardize inputs so angles $\in [-\pi, \pi]$. With $> \sim 8$ features, do feature selection/PCA first.

Hybrid Interface Pattern

Decoding (Circuit $\rightarrow$ Numbers)

- **Expectations:** $\hat{p} = (1 - \langle Z_0 \rangle)/2$ (binary); $f(x) = \sum_i w_i \langle Z_i \rangle$ (regression).
- **Probabilities:** bitstring histogram for uncertainty.
- **Kernels:** $K(x, x') = |\langle \phi(x) | \phi(x') \rangle|^2$ for SVM fallback.

Optimization Loop

- **Parameter-shift:**
$$\partial_{\theta_k} \langle H \rangle = \frac{1}{2} \left(\langle H \rangle_{\theta_k + \frac{\pi}{2}} - \langle H \rangle_{\theta_k - \frac{\pi}{2}} \right)$$

(≈ 2 evals/parameter $\times$ shots).
- **Gradient-free:** **SPSA** (2 evals/step, dim-independent), **COBYLA/Nelder-Mead** (few params).
- **Batching & caching:** batch inputs per θ ; reuse transpiled circuits; group commuting Pauli terms.

Where Quantum Fits in Deep Nets

Placement patterns

- **Bottleneck head (most common):** ... $\rightarrow$ Dense(k) $\rightarrow$ QuantumLayer(m) $\rightarrow$ Dense/Softmax
($k \approx 2-8$ features $\rightarrow n \approx k$ qubits $\rightarrow m \approx 1-4$ expectations).
- **Mid-stack fusion:** quantum layer as nonlinear mixer; concat or residual-add back into classical stream.
- **Lightweight QCNN vibe:** small repeated quantum blocks interleaved with classical pooling/conv.
- **Attention hooks:** use quantum layer to produce keys/queries in low dimension.

Where Quantum Fits in Deep Nets

Rules of thumb

- Keep qubits/features small (2–8).
- Start with **1–2 reps** of RY/RZ + CX aligned to coupling map.
- Small-variance init (e.g., $N(0, 0.1)$) to reduce barren plateaus.
- Normalize outputs (LayerNorm) before/after quantum layer when feeding dense stacks.

Tooling and APIs

Qiskit + PyTorch (recommended): Primitives (Estimator/Sampler) →

`EstimatorQNN` / `SamplerQNN` → `TorchConnector` for autograd-friendly layers.

TensorFlow/Keras:

- *Native:* TensorFlow Quantum (`tfq.layers.PQC`) as Keras `Layer` .
- *Pragmatic with Qiskit:* a wrapper layer using `tf.numpy_function` (non-autodiff) or two-phase training (train quantum in PyTorch, freeze in Keras).

PennyLane: Device-agnostic; `qml.qnn.TorchLayer` / `KerasLayer` turn QNodes into DL layers.

CUDA-Q: HPC-friendly simulation + PyTorch integration for heavy training loops.

Code Templates

```
1 # Qiskit 2.x circuit builders
2 from qiskit.circuit.library import zz_feature_map, real_amplitudes, efficient_su2
3 from qiskit import QuantumCircuit
4
5 # Example: 4-feature entangling encoder + shallow ansatz
6 fm = zz_feature_map(feature_dimension=4, reps=1, entanglement="linear") # QuantumCircuit
7 ans1 = real_amplitudes(num_qubits=4, reps=1, entanglement="linear")
8 # or:
9 ans2 = efficient_su2(num_qubits=4, reps=1, entanglement="linear")
10
11 qc = fm.compose(ans1) # combined model circuit
12 |
```

Primitives (Simulator vs Hardware)

```
1 from qiskit_aer.primitives import Estimator, Sampler # local backends
2 # Use Estimator for expectations, Sampler for bitstring distributions.
```

```
~ |
```

```
1 # pip install qiskit-ibm-runtime
2 from qiskit_ibm_runtime import QiskitRuntimeService, EstimatorV2 as Estimator, SamplerV2 as Sampler
3
4 service = QiskitRuntimeService() # requires saved account
5 estimator = Estimator() # V2 primitive
6 # Run with PUB tuples: (circuit, observables, parameter_values)
7 # For example:
8 # job = estimator.run([ (qc, [obs1, obs2], param_values_dict) ], precision=0.05)
9 # res = job.result()
```

```
1 # pip install qiskit qiskit-aer qiskit-machine-learning torch scikit-learn
2 import torch, torch.nn as nn
3 from qiskit.circuit.library import zz_feature_map, real_amplitudes
4 from qiskit_aer.primitives import Estimator
5 from qiskit_machine_learning.neural_networks import EstimatorQNN
6 from qiskit_machine_learning.connectors import TorchConnector
```

```

8 # 1) 4-dim bottleneck -> 4-qubit encoder + shallow ansatz
9 fm = zz_feature_map(feature_dimension=4, reps=1, entanglement="linear")
10 ans = real_amplitudes(num_qubits=4, reps=1, entanglement="linear")
11 model_circ = fm.compose(ans)
12
13 qnn = EstimatorQNN(
14     circuit=model_circ,
15     input_params=fm.parameters,      # data-bound symbols
16     weight_params=ans.parameters,    # trainable symbols
17     estimator=Estimator(shots=2048) # local sim; swap to Runtime EstimatorV2 for hardware
18 )
19 QuantumLayer = TorchConnector(qnn) # nn.Module that maps [B,4] -> [B,1] expectation
20

```

```
21 class HybridNet(nn.Module):
22     def __init__(self, in_dim=64, out_dim=1):
23         super().__init__()
24         self.pre = nn.Sequential(nn.Linear(in_dim,16), nn.ReLU(), nn.Linear(16,4))
25         self.q = QuantumLayer
26         self.post = nn.Sequential(nn.Linear(1,8), nn.ReLU(), nn.Linear(8,out_dim))
27     def forward(self, x):
28         z = self.pre(x)
29         q = self.q(z)          # [B,1]
30         return self.post(q)
31
```

Patterns and when to use

- **Quantum feature head** → **classical body**: tabular, small medical/materials datasets.
- **Classical extractor** → **quantum head**: small CNN → 4–6 dims → PQC decision.
- **Kernel fallback**: keep feature map, switch to quantum kernel SVM if gradients collapse.
- **Subroutines**: QAOA/VQE for discrete inner loops.

4.3 TensorFlow/Keras (native route)

Use **TensorFlow Quantum**: `tfq.layers.PQC` / `ControlledPQC` are Keras-native, giving PQC layers with TF autodiff and optimizers. (Recommended if your team is TF-first.)

IBM Quantum D...

If you must stay in Qiskit for TF, wrap the quantum call in a `tf.keras.layers.Layer` with a custom gradient (or do two-phase training—train quantum in PyTorch, then freeze it in Keras).

4.4 PennyLane (device-agnostic)

`qml.qnn.TorchLayer` / `KerasLayer` convert QNodes into DL layers with native autodiff; useful for rapid prototyping and swapping simulators/hardware.

- **Feature count:** aim 2–8 features → 2–8 qubits; do PCA/top-k first.
- **Topology-aware entanglers:** match coupling map to avoid SWAPs (shallower, less error).
- **Mitigation:** readout calibration/mitigation improves expectations feeding losses; add DD on idles; use ZNE at checkpoints (not each step). qiskit-communit...
- **Shots schedule:** 256–512 early, 2k–8k near convergence.
- **Latency:** use **sessions** and batch many parameter sets/input batches per call; cache transpiled circuits.
- **APIs that age well:** function builders (`zz_feature_map` , `real_amplitudes` , `efficient_su2`) + **Runtime V2** for hardware. IBM Quantum Cl... +1

```
1 # pip install qiskit qiskit-aer qiskit-machine-learning torch scikit-learn
2 import torch, torch.nn as nn
3 from sklearn.datasets import make_moons
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler
6
7 from qiskit.circuit.library import zz_feature_map, real_amplitudes
8 from qiskit_aer.primitives import Estimator
9 from qiskit_machine_learning.neural_networks import EstimatorQNN
10 from qiskit_machine_learning.connectors import TorchConnector
11
```

```
12 # 1) data
13 X, y = make_moons(n_samples=240, noise=0.12, random_state=0)
14 X = StandardScaler().fit_transform(X)
15 Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0)
16 Xtr = torch.tensor(Xtr, dtype=torch.float32); ytr = torch.tensor(ytr, dtype=torch.float32)
17 Xte = torch.tensor(Xte, dtype=torch.float32); yte = torch.tensor(yte, dtype=torch.float32)
```

```
19 # 2) circuit (2 qubits)
20 fm = zz_feature_map(feature_dimension=2, reps=1, entanglement="linear")
21 ans = real_amplitudes(num_qubits=2, reps=2, entanglement="linear")
22 qc = fm.compose(ans)
```

```
24 # 3) quantum network + torch bridge
25 qnn = EstimatorQNN(circuit=qc, input_params=fm.parameters,
26                    weight_params=ans.parameters, estimator=Estimator(shots=1024))
27 QuantumLayer = TorchConnector(qnn) # -> [B,1]
```

```
29 # 4) small hybrid model
30 class VQCNet(nn.Module):
31     def __init__(self):
32         super().__init__()
33         self.q = QuantumLayer
34         self.head = nn.Sequential(nn.Sigmoid()) # for BCE
35     def forward(self, x):
36         return self.head(self.q(x)).squeeze(-1)
```

```
42 # 5) train (shots can increase later)
43 for epoch in range(60):
44     opt.zero_grad()
45     pred = model(Xtr)
46     loss = lossf(pred, ytr)
47     loss.backward()
48     opt.step()
```

```
50 with torch.no_grad():
51     acc = ((model(Xte)>0.5) == yte.bool()).float().mean().item()
52 print(f"Test accuracy: {acc:.3f}")
53 |
```


Debugging

- **Flat loss?** Angles saturating → re-scale inputs; check measurement mapping.
- **Barren plateaus?** Too deep/random ansatz → fewer reps, structured encoding, good init, layer-wise growth.
- **Latency spikes?** Missing batching/sessions or recompiling each step → reuse transpiled circuits; batch parameter sets.
- **Hardware mismatch?** SWAP bloat → topology-aware entanglers; set an initial layout.
- **Noisy expectations?** Turn on readout mitigation; consider DD; increase shots near convergence. qiskit-communit...

Use Qiskit 2.1+ Conventions (functions, not classes)

```
1 from qiskit.circuit.library import zz_feature_map, real_amplitudes, efficient_su2
2 from qiskit.quantum_info import SparsePauliOp
3 from qiskit import QuantumCircuit
4
5 # 4-feature encoder (entangling angle map) + shallow ansätze
6 fm = zz_feature_map(feature_dimension=4, reps=1, entanglement="linear")
7 ans1 = real_amplitudes(num_qubits=4, reps=1, entanglement="linear")
8 ans2 = efficient_su2(num_qubits=4, reps=1, entanglement="linear")
9
10 qc = fm.compose(ans1) # combined model circuit
11
12 # Observables (Pauli sums)
13 H = SparsePauliOp.from_list([
14     ("ZZII", -0.7), ("IZZI", -0.7), ("IIZZ", -0.7), ("XXXX", 0.3)
15 ])
16
```

B1) Parameter-shift: quick derivation sketch

For a gate $U(\theta_k) = \exp(-i \theta_k P/2)$ with Pauli $P^2 = I$,

$$\frac{\partial}{\partial \theta_k} \langle H \rangle_{\theta} = \frac{1}{2} \left(\langle H \rangle_{\theta_k + \frac{\pi}{2}} - \langle H \rangle_{\theta_k - \frac{\pi}{2}} \right),$$

because $U(\theta_k \pm \frac{\pi}{2})$ rotates the eigencomponents of P by $\pm \pi/2$, yielding an exactly centered finite difference.

Vector case: For $\theta \in \mathbb{R}^p$, you need **two evaluations per parameter** (holding all others fixed), so a full gradient costs $\approx 2p$ **evaluations** (times the number of distinct observables).

B2) Cost accounting and batching

Let:

- N = samples per epoch (or per gradient step if full-batch),
- p = number of trainable parameters,
- S = shots per expectation evaluation,
- M = number of measured observables per forward pass (often 1–4).

Full-batch parameter-shift:

Evaluations/epoch $\approx N \times 2p$ (each evaluation can process a **batch** of inputs at once if your primitive run supports vectorized params).

Mini-batch B : Evaluations/epoch $\approx \frac{N}{B} \times 2p$.

Always **batch inputs** for a fixed θ to amortize latency; send **many parameter vectors** per `run()`.

B3) Shot noise & standard error (SE)

For a single-qubit Pauli expectation $\mu = \langle Z \rangle$ estimated from S Bernoulli trials (treating $+1/-1$ as outcomes), the standard error is

$$\text{SE}(\hat{\mu}) = \sqrt{\frac{1 - \mu^2}{S}} \quad (\text{worst case at } \mu = 0 : \text{SE} = 1/\sqrt{S}).$$

Targets

- If you want $\text{SE} \leq 0.05$ in the **worst case**, choose $S \geq (1/0.05)^2 = 400$ **per expectation**.
- For BCE training where you need **probability** $\hat{p} = (1 - \hat{\mu})/2$, the SE halves:
 $\text{SE}(\hat{p}) = \text{SE}(\hat{\mu})/2$.

B4) Multi-term observables

If $H = \sum_{i=1}^M w_i P_i$ (non-commuting generally measured in separate groups),

$$\hat{E} = \sum_{i=1}^M w_i \hat{\mu}_i, \quad \text{Var}(\hat{E}) = \sum_{i=1}^M w_i^2 \text{Var}(\hat{\mu}_i)$$

assuming independent estimates per term/group. To hit a **target SE** on $\hat{E}$, you can:

- **Uniform shots** $S_i = S$ across groups, or
- **Importance allocation** $S_i \propto |w_i| \sqrt{\text{Var}(\hat{\mu}_i)}$ (empirical), reducing total shots.

B5) SPSA vs parameter-shift (budget intuition)

Method	Evals/step	Scales with p ?	Noise tolerance	Notes
Param-shift	$2p$	Yes (linear)	Good if S large	Exact grads for many gates
SPSA	2	No	Very robust	Tunable gain sequences; great early on

Practical recipe

- **Exploration:** use **SPSA** with low shots (256–512).
- **Convergence:** switch to **parameter-shift** and raise shots (2k–8k), or keep SPSA and **average** more shot-batches.

B6) Worked budgeting example

- Binary classifier with 2 qubits, $p = 12$ params, $M = 1$ observable.
- Dataset $N = 200$, mini-batch $B = 20 \rightarrow 10$ batches/epoch.
- Parameter-shift: evaluations/epoch = $10 \times 2p = 240$.
- With $S = 512$ shots each $\rightarrow \sim 123,000$ total shots/epoch for one observable.
- If you add a second observable head ($M = 2$) measured in a separate group $\rightarrow$ double the measurement calls or use commuting grouping where possible.